

Unit-4 (mp)

Javabeans - Acc. to Java white paper, it is a reusable software component. A bean encapsulates many objects in one object so that are access this object from multiple places, moreover, it provides easy maintenance.

- ↳ Reusability is the main concept in any programming language.
- ↳ Javabeans is a portable, platform independent model written in Java programming language.
- ↳ Javabeans contain several elements like constructor, Getter/setter methods.

Preparing a class to be a Javabeans:

- ↳ It should have a no-arg constructor - default constructor.
- ↳ It should be serializable. Beans should implement `java.io.Serializable` as it allow to save, store and restore the state of a Javabeans you are working

as.

ex-

```
public class TestBean {
    private String name;
    public void setName(String name)
    {
        this.name = name;
    }
    public String getName()
    {
        return name;
    }
}
```

Setter method: It should be Public in nature

- ↳ The return-type should be void
- ↳ It should take some argument

Getter method: It should be Public in nature

- ↳ The return-type should not be void.
- ↳ It should not take any argument.

Creating a Java Beans -

- ↳ implements `java.io.Serializable`
- ↳ Declare Private variable

- History of Node.js
- Initially, JS was used in frontend only, to enhance UI and interaction on website
 - Node.js allow developer to use same language in frontend and Backend i.e. client side as well as server side
 - Node.js was developed by Ryan Dahl
 - Node.js is javascript runtime build on chrome V8 JS engine.

Vs Engine

- developed by Google

Human readable code $\xrightarrow[\text{engine}]{\text{Vs}}$ Machine Readable code

- Node.js is a cross-platform, open-source maintained by (open JS foundation) JS runtime environment that can run on "window", "Linux" or more

- Node.js help us to run JS outside of the browser

History of Node.js

① 2009 → Ryan Dahl started Node.js

↳ Initially started on Firefox

② 2010 → NPM was introduced

③ 2011 → window support for Node.js

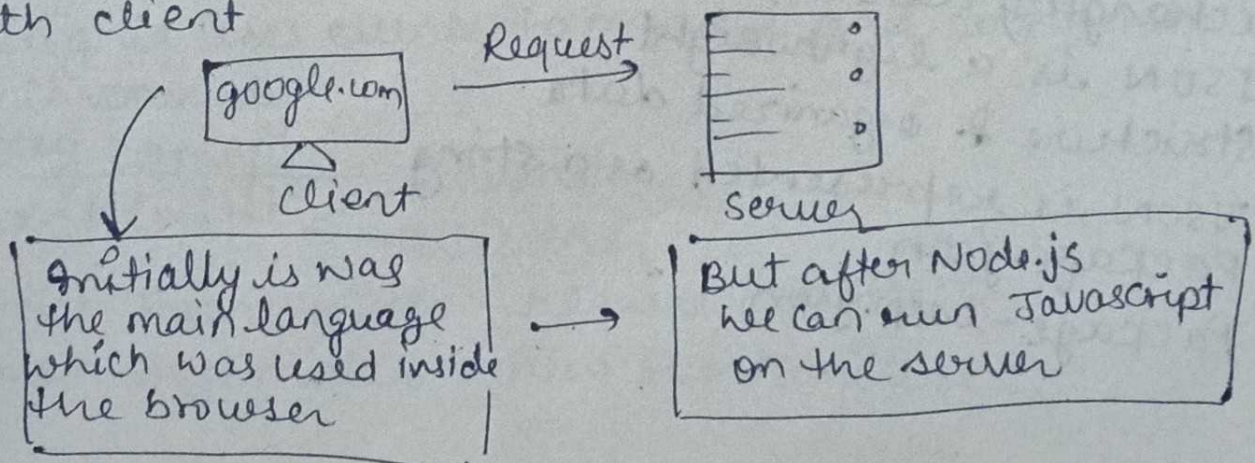
④ 2012 → Ryan left Node.js team

↳ Isaac become the new leader

⑤ 2014 → bc of the slow pace of development in Node.js

⑥ 2015 → Fedor and Joyent merged and Node.js foundation

Server - A server is a person who communicate with client



... is a bridge that allows two different software application to talk to each other

- API are used to access data or functionality on your server.

ex → collection of list = menu card = API

Endpoints - is a specific URL on your server that represents one operations or data access point in your API.

ex - option in that list == endpoints

Callback concept: is crucial for handling tasks that take time to complete, like reading a file, querying a data use, or making an API request

- Node.js is asynchronous and non-blocking by default
- for ex - ^{file system}

```
const fs = require('fs');
```

```
fs.readFile('example.txt', 'UTF8', (err, data)
```

```
  {  
    if (err) {
```

```
      console.log('error reading:', err);
```

```
      return;
```

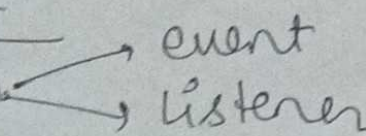
```
    } console.log('file content:', data);
```

```
  } })
```

events in Node.js -

- every action on a computer is an event like when a connection is made or a file is opened.
- Node.js is built on an event-driven architecture, meaning it uses events to manage and coordinate asynchronously.

Key Concept

① event emitter 

```
const EventEmitter = require('events');
```

```
const myEmitter = new EventEmitter();
```